AdditionalCommitNotes

Perfect — this is exactly the right way to handle a hardware baseline.

You want:

- New repository name: **piStepperWorkingBaseline**
- Based on commit: `24ac681`
- Completely independent from your current repo
- Clean and permanent for regression testing

We'll do this cleanly and safely.

✅ OVERVIEW OF WHAT WE'LL DO

1. Checkout commit `24ac681`
2. Create a new branch from it
3. Remove connection to old GitHub repo
4. Create a new GitHub repository
5. Push this baseline to the new repo

📋 COPY-PASTE COMMAND BLOCK (for Obsidian)

Assuming you are inside your existing repo directory:

```
# 1. Go to repo root (if not already there)
cd /path/to/piStepperMotorControl

# 2. Make sure working tree is clean
git status

# 3. Checkout the known-good commit
git checkout 24ac681

# 4. Create a proper branch from it
git checkout -b baseline-24ac681

# 5. Rename the directory (optional but recommended)
```

```
cd ..
mv piStepperMotorControl piStepperWorkingBaseline
cd piStepperWorkingBaseline

# 6. Remove old GitHub remote
git remote remove origin

# 7. Create NEW repo on GitHub:
#    Name: piStepperWorkingBaseline
#    DO NOT initialize with README
#    DO NOT add .gitignore
#    DO NOT add license

# 8. Add new remote (replace with your actual URL)
git remote add origin
https://github.com/RobSteele49/piStepperWorkingBaseline.git

# 9. Push to GitHub
git push -u origin baseline-24ac681
```

🟢 OPTIONAL: Make `main` the branch name

If you'd prefer the new repo to have `main` instead of `baseline-24ac681`:

After step 4, do this instead:

```
git checkout -b main
```

Then push:

```
git push -u origin main
```

That makes this commit the official main branch of the new repository.

🧠 What This Gives You

You now have:

Repo 1:

```
piStepperMotorControl
```

→ Contains UI experiments

Repo 2:

```
piStepperWorkingBaseline
```

→ Frozen hardware-verified baseline
→ Safe for regression testing
→ Can always compare against

🔒 VERY IMPORTANT

You now have two completely independent repositories.

Changes in one will NOT affect the other.

This is exactly what you want for hardware projects.

⭐ Bonus (Highly Recommended)

Tag the baseline inside the new repo:

```
git tag hardware-verified-initial
git push origin hardware-verified-initial
```

Now you have a permanent marker.

If you'd like next, I can show you a professional workflow where:

- Baseline repo stays clean
- Development happens in feature branches
- You can merge UI fixes once hardware parity is confirmed

This is exactly how embedded teams structure production systems.